

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием RBM»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверил:

Никитин С.К.

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью RBM.

Ход работы

1. Взять за основу нейронную сеть из лабораторной работы №3. Выполнить обучение с предобучением, используя стек ограниченных машин Больцмана (RBM – Restricted Boltzmann Machine), алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев как RBM выбрать самостоятельно.
2. Сравнить результаты, полученные при
 - обучении без предобучения (ЛР 3);
 - обучении с предобучением, используя автоэнкодерный подход (ЛР3);
 - обучении с предобучением, используя RBM.
3. Обучить модели на данных из ЛР 2, сравнить результаты по схеме из пункта 2;
4. Сделать выводы, оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Тип задачи	Целевая переменная
9	https://archive.ics.uci.edu/dataset/850/raisin	Классификация	Class

В работе взяли нейронную сеть из лабораторной работы №3 и выполнили её предобучение с помощью стека ограниченных машин Больцмана. Для датасета Raisin использовали архитектуру $7 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 1$, для датасета Wine Quality (white) — $11 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 4$. Предобучение выполнялось послойно через BernoulliRBM из scikit-learn: сначала обучалась RBM $7 \rightarrow 32$, затем RBM $32 \rightarrow 16$ для Raisin, и аналогично $11 \rightarrow 64$ и $64 \rightarrow 32$ для Wine. Условие останова выбрано самостоятельно — 50 итераций на каждый слой. Полученные веса RBM (компоненты и смещения скрытого слоя) перенесли в соответствующие слои Dense классификатора, после чего модели дообучили на размеченных данных. Затем сравнили три схемы обучения: baseline без предобучения, с предобучением через автоэнкодер и с предобучением через RBM. Для каждого датасета построили таблицы с метриками (Accuracy, Precision, Recall, F1 для Raisin; Accuracy, F1_macro, F1_weighted для Wine), добавили колонки Δ_{AE} и Δ_{RBM} относительно baseline и построили три матрицы ошибок. Схему из пункта 2 повторили на датасете из ЛР2 — Wine Quality (white) с той же группировкой в четыре класса (Low 3-4, Medium 5, High 6, Very High 7-9). Для RBM-ветки признаки нормализовали через MinMaxScaler в диапазон [0, 1], поскольку BernoulliRBM работает с вероятностями активаций, а для baseline и AE использовался StandardScaler. В конце программа автоматически формирует выводы на основе фактических значений метрик, а все графики сохраняются в файлы lr4_raisin_comparison.png и lr4_wine_comparison.png.

Код программы:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import torch
import torch.nn as nn
import tensorflow as tf
from tensorflow.keras import layers, Model
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix)

import warnings
warnings.filterwarnings('ignore')

np.random.seed(42)
torch.manual_seed(42)
tf.random.set_seed(42)

class RBM(nn.Module):
    def __init__(self, n_visible, n_hidden):
        super().__init__()
        self.W = nn.Parameter(torch.randn(n_visible, n_hidden) * 0.01)
        self.bv = nn.Parameter(torch.zeros(n_visible))
        self.bh = nn.Parameter(torch.zeros(n_hidden))

    def sample_h(self, v):
        p_h = torch.sigmoid(v @ self.W + self.bh)
        return p_h

    def sample_v(self, h):
        p_v = torch.sigmoid(h @ self.W.t() + self.bv)
        return p_v

    def free_energy(self, v):
        vbias_term = v @ self.bv
        wx_b = v @ self.W + self.bh
        hidden_term = torch.log(1 + torch.exp(wx_b)).sum(dim=1)
        return -hidden_term - vbias_term

    def cd_loss(self, v0, k=1):
        p_h0 = self.sample_h(v0)
        h0 = torch.bernoulli(p_h0)
        hk = h0
        for _ in range(k):
            p_vk = self.sample_v(hk)
            vk = torch.bernoulli(p_vk)
            p_hk = self.sample_h(vk)
            hk = torch.bernoulli(p_hk)
        loss = self.free_energy(v0).mean() - self.free_energy(vk).mean()
        return loss

def train_rbm(X_data, n_hidden, epochs=100, lr=0.05, batch_size=64, k=1):
    X_tensor = torch.FloatTensor(X_data)
    n_visible = X_data.shape[1]
    rbm = RBM(n_visible, n_hidden)
    optimizer = torch.optim.SGD(rbm.parameters(), lr=lr, momentum=0.9)

    n = X_tensor.size(0)
    for epoch in range(epochs):
        perm = torch.randperm(n)
```

```

X_shuffled = X_tensor[perm]
epoch_loss = 0.0
for i in range(0, n, batch_size):
    batch = X_shuffled[i:i + batch_size]
    loss = rbm.cd_loss(batch, k=k)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    epoch_loss += loss.item() * batch.size(0)
epoch_loss /= n
if (epoch + 1) % 20 == 0:
    print(f"    RBM epoch {epoch+1}/{epochs}, loss={epoch_loss:.4f}")
return rbm

def rbm_hidden_probs(rbm, X_data):
    X_tensor = torch.FloatTensor(X_data)
    with torch.no_grad():
        p_h = rbm.sample_h(X_tensor)
    return p_h.numpy()

def set_dense_weights_from_rbm(dense_layer, rbm):
    W = rbm.W.detach().numpy().astype(np.float32)
    bh = rbm.bh.detach().numpy().astype(np.float32)
    dense_layer.set_weights([W, bh])

def binarize(X):
    med = np.median(X, axis=0)
    return (X > med).astype(np.float32)

print("\n" + "#" * 60)
print("#  ЧАСТЬ A. RAISIN")
print("#" * 60)

df_r = pd.read_excel("Raisin_Dataset.xlsx")
X_r = df_r.drop("Class", axis=1).values
y_r = LabelEncoder().fit_transform(df_r["Class"].values)

scaler_r = MinMaxScaler()
X_r_scaled = scaler_r.fit_transform(X_r).astype(np.float32)
X_r_bin = binarize(X_r_scaled)

Xr_tr, Xr_te, yr_tr, yr_te = train_test_split(
    X_r_scaled, y_r, test_size=0.2, random_state=42, stratify=y_r
)

input_dim_r = X_r_scaled.shape[1]

def build_clf_r(input_dim, hidden=(32, 16, 8)):
    m = tf.keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(hidden[0], activation='relu'),
        layers.Dense(hidden[1], activation='relu'),
        layers.Dense(hidden[2], activation='relu'),
        layers.Dense(1, activation='sigmoid')
    ])
    m.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return m

def clf_report(y_true, y_pred, name):
    print(f"[{name}] Acc={accuracy_score(y_true,y_pred):.4f} "
          f"Prec={precision_score(y_true,y_pred):.4f} "
          f"Rec={recall_score(y_true,y_pred):.4f} "
          f"F1={f1_score(y_true,y_pred):.4f}")

```

```

print("\n--- Raisin: Baseline ---")
base_r = build_clf_r(input_dim_r)
base_r.fit(Xr_tr, yr_tr, epochs=100, batch_size=32,
           validation_split=0.1, verbose=0)
y_base_r = (base_r.predict(Xr_te, verbose=0) > 0.5).astype(int)
clf_report(yr_te, y_base_r, "Raisin Baseline")

def pretrain_ae(data_in, hidden_dim, epochs=50):
    inp = layers.Input(shape=(data_in.shape[1],))
    enc = layers.Dense(hidden_dim, activation='relu')(inp)
    dec = layers.Dense(data_in.shape[1], activation='linear')(enc)
    ae = Model(inp, dec)
    ae.compile(optimizer='adam', loss='mse')
    ae.fit(data_in, data_in, epochs=epochs, batch_size=32, verbose=0)
    return Model(inp, enc)

print("\n--- Raisin: AE Pretraining ---")
ae1_r = pretrain_ae(X_r_scaled, 32, 50)
H1_r = ae1_r.predict(X_r_scaled, verbose=0)
ae2_r = pretrain_ae(H1_r, 16, 50)

pre_ae_r = build_clf_r(input_dim_r)
pre_ae_r.layers[0].set_weights(ae1_r.get_weights())
pre_ae_r.layers[1].set_weights(ae2_r.get_weights())
pre_ae_r.fit(Xr_tr, yr_tr, epochs=100, batch_size=32,
             validation_split=0.1, verbose=0)
y_ae_r = (pre_ae_r.predict(Xr_te, verbose=0) > 0.5).astype(int)
clf_report(yr_te, y_ae_r, "Raisin AE")

print("\n--- Raisin: RBM Pretraining ---")
rbm1_r = train_rbm(X_r_bin, n_hidden=32, epochs=100, lr=0.05, batch_size=64, k=1)
H1_rbm_r = rbm_hidden_probs(rbm1_r, X_r_bin)

rbm2_r = train_rbm(H1_rbm_r, n_hidden=16, epochs=100, lr=0.05, batch_size=64, k=1)

pre_rbm_r = build_clf_r(input_dim_r)
set_dense_weights_from_rbm(pre_rbm_r.layers[0], rbm1_r)
set_dense_weights_from_rbm(pre_rbm_r.layers[1], rbm2_r)

pre_rbm_r.fit(Xr_tr, yr_tr, epochs=100, batch_size=32,
              validation_split=0.1, verbose=0)
y_rbm_r = (pre_rbm_r.predict(Xr_te, verbose=0) > 0.5).astype(int)
clf_report(yr_te, y_rbm_r, "Raisin RBM")

print("\n--- Raisin: Сравнение ---")
comp_r = pd.DataFrame({
    'Метрика': ['Accuracy', 'Precision', 'Recall', 'F1'],
    'Baseline': [accuracy_score(yr_te, y_base_r),
                  precision_score(yr_te, y_base_r),
                  recall_score(yr_te, y_base_r),
                  f1_score(yr_te, y_base_r)],
    'AE': [accuracy_score(yr_te, y_ae_r),
           precision_score(yr_te, y_ae_r),
           recall_score(yr_te, y_ae_r),
           f1_score(yr_te, y_ae_r)],
    'RBM': [accuracy_score(yr_te, y_rbm_r),
            precision_score(yr_te, y_rbm_r),
            recall_score(yr_te, y_rbm_r),
            f1_score(yr_te, y_rbm_r)]
})
comp_r[['Baseline', 'AE', 'RBM']] = comp_r[['Baseline', 'AE', 'RBM']].round(4)
comp_r['Δ_AE'] = (comp_r['AE'] - comp_r['Baseline']).round(4)
comp_r['Δ_RBM'] = (comp_r['RBM'] - comp_r['Baseline']).round(4)
print(comp_r.to_string(index=False))

```

```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, pred, title in zip(axes, [y_base_r, y_ae_r, y_rbm_r],
                             ['Baseline', 'AE', 'RBM']):
    sns.heatmap(confusion_matrix(yr_te, pred), annot=True, fmt='d',
                  cmap='Blues', ax=ax,
                  xticklabels=['Kecimen', 'Besni'],
                  yticklabels=['Kecimen', 'Besni'])
    ax.set_title(f'Raisin - {title}')
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
plt.tight_layout()
plt.savefig('lr4_raisin_comparison.png', dpi=120)
plt.show()

print("\n" + "#" * 60)
print("# ҒАҒТЫ Б. WINE QUALITY (WHITE)")
print("#" * 60)

url_white = ("https://archive.ics.uci.edu/ml/machine-learning-databases/"
             "wine-quality/winequality-white.csv")
white_wine = pd.read_csv(url_white, sep=';')

X_w = white_wine.drop(columns=['quality']).values
y_w_raw = white_wine['quality'].values

def group_quality(q):
    if q <= 4:
        return 0
    elif q == 5:
        return 1
    elif q == 6:
        return 2
    else:
        return 3

y_w = np.array([group_quality(q) for q in y_w_raw])
class_names = {0: 'Low (3-4)', 1: 'Medium (5)', 2: 'High (6)', 3: 'Very High (7-9)'}

scaler_w = MinMaxScaler()
X_w_scaled = scaler_w.fit_transform(X_w).astype(np.float32)
X_w_bin = binarize(X_w_scaled)

Xw_tr, Xw_te, yw_tr, yw_te = train_test_split(
    X_w_scaled, y_w, test_size=0.2, random_state=42, stratify=y_w
)

input_dim_w = X_w_scaled.shape[1]

def build_clf_w(input_dim, n_classes=4, hidden=(64, 32, 16)):
    m = tf.keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(hidden[0], activation='relu'),
        layers.Dense(hidden[1], activation='relu'),
        layers.Dense(hidden[2], activation='relu'),
        layers.Dense(n_classes, activation='softmax')
    ])
    m.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m

def multi_report(y_true, y_pred, name):
    print(f"[{name}] Acc={accuracy_score(y_true, y_pred):.4f} "
          f"F1_macro={f1_score(y_true, y_pred, average='macro'):.4f} "
          f"F1_weighted={f1_score(y_true, y_pred, average='weighted'):.4f}")

```

```

print("\n--- Wine: Baseline ---")
base_w = build_clf_w(input_dim_w, n_classes=4)
base_w.fit(Xw_tr, yw_tr, epochs=100, batch_size=64,
           validation_split=0.1, verbose=0)
y_base_w = np.argmax(base_w.predict(Xw_te, verbose=0), axis=1)
multi_report(yw_te, y_base_w, "Wine Baseline")

print("\n--- Wine: AE Pretraining ---")
a1_w = pretrain_ae(X_w_scaled, 64, 50)
H1_w = a1_w.predict(X_w_scaled, verbose=0)
ae2_w = pretrain_ae(H1_w, 32, 50)

pre_ae_w = build_clf_w(input_dim_w, n_classes=4)
pre_ae_w.layers[0].set_weights(a1_w.get_weights())
pre_ae_w.layers[1].set_weights(ae2_w.get_weights())
pre_ae_w.fit(Xw_tr, yw_tr, epochs=100, batch_size=64,
            validation_split=0.1, verbose=0)
y_ae_w = np.argmax(pre_ae_w.predict(Xw_te, verbose=0), axis=1)
multi_report(yw_te, y_ae_w, "Wine AE")

print("\n--- Wine: RBM Pretraining ---")
rbm1_w = train_rbm(X_w_bin, n_hidden=64, epochs=100, lr=0.05, batch_size=64, k=1)
H1_rbm_w = rbm_hidden_probs(rbm1_w, X_w_bin)

rbm2_w = train_rbm(H1_rbm_w, n_hidden=32, epochs=100, lr=0.05, batch_size=64, k=1)

pre_rbm_w = build_clf_w(input_dim_w, n_classes=4)
set_dense_weights_from_rbm(pre_rbm_w.layers[0], rbm1_w)
set_dense_weights_from_rbm(pre_rbm_w.layers[1], rbm2_w)

pre_rbm_w.fit(Xw_tr, yw_tr, epochs=100, batch_size=64,
             validation_split=0.1, verbose=0)
y_rbm_w = np.argmax(pre_rbm_w.predict(Xw_te, verbose=0), axis=1)
multi_report(yw_te, y_rbm_w, "Wine RBM")

print("\n--- Wine: Сравнение ---")
comp_w = pd.DataFrame({
    'Метрика': ['Accuracy', 'F1_macro', 'F1_weighted'],
    'Baseline': [accuracy_score(yw_te, y_base_w),
                 f1_score(yw_te, y_base_w, average='macro'),
                 f1_score(yw_te, y_base_w, average='weighted')],
    'AE': [accuracy_score(yw_te, y_ae_w),
           f1_score(yw_te, y_ae_w, average='macro'),
           f1_score(yw_te, y_ae_w, average='weighted')],
    'RBM': [accuracy_score(yw_te, y_rbm_w),
            f1_score(yw_te, y_rbm_w, average='macro'),
            f1_score(yw_te, y_rbm_w, average='weighted')]
})
comp_w[['Baseline', 'AE', 'RBM']] = comp_w[['Baseline', 'AE', 'RBM']].round(4)
comp_w['Δ_AE'] = (comp_w['AE'] - comp_w['Baseline']).round(4)
comp_w['Δ_RBM'] = (comp_w['RBM'] - comp_w['Baseline']).round(4)
print(comp_w.to_string(index=False))

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, pred, title in zip(axes, [y_base_w, y_ae_w, y_rbm_w],
                           ['Baseline', 'AE', 'RBM']):
    sns.heatmap(confusion_matrix(yw_te, pred), annot=True, fmt='d',
                cmap='Blues', ax=ax,
                xticklabels=list(class_names.values()),
                yticklabels=list(class_names.values()))
    ax.set_title(f'Wine - {title}')
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
plt.tight_layout()
plt.savefig('lr4_wine_comparison.png', dpi=120)

```

```

plt.show()

print("\n" + "=" * 60)
print("ВЫВОДЫ")
print("=" * 60)

d_ae_r = fl_score(yr_te, y_ae_r) - fl_score(yr_te, y_base_r)
d_rbm_r = fl_score(yr_te, y_rbm_r) - fl_score(yr_te, y_base_r)

print(f"\nRAISIN:")
print(f"Baseline F1={fl_score(yr_te, y_base_r):.4f}")
print(f"AE      F1={fl_score(yr_te, y_ae_r):.4f}  (Δ={d_ae_r:+.4f})")
print(f"RBM      F1={fl_score(yr_te, y_rbm_r):.4f}  (Δ={d_rbm_r:+.4f})")

d_ae_w = (fl_score(yw_te, y_ae_w, average='macro')
          - fl_score(yw_te, y_base_w, average='macro'))
d_rbm_w = (fl_score(yw_te, y_rbm_w, average='macro')
          - fl_score(yw_te, y_base_w, average='macro'))

print(f"\nWINE QUALITY (WHITE):")
print(f"Baseline F1_macro={fl_score(yw_te, y_base_w, average='macro'):.4f}")
print(f"AE      F1_macro={fl_score(yw_te, y_ae_w, average='macro'):.4f}  (Δ={d_ae_w:+.4f})")
print(f"RBM      F1_macro={fl_score(yw_te, y_rbm_w, average='macro'):.4f}  (Δ={d_rbm_w:+.4f})")

```

Результат программы:

ЧАСТЬ А. RAISIN

--- Raisin: Baseline ---

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

[Raisin Baseline] Acc=0.8833 Prec=0.8485 Rec=0.9333 F1=0.8889

--- Raisin: AE Pretraining ---

[Raisin AE] Acc=0.8833 Prec=0.8485 Rec=0.9333 F1=0.8889

--- Raisin: RBM Pretraining ---

RBM epoch 20/100, loss=-0.0304

RBM epoch 40/100, loss=-0.0597

RBM epoch 60/100, loss=-0.0289

RBM epoch 80/100, loss=0.1038

RBM epoch 100/100, loss=0.0358

RBM epoch 20/100, loss=0.2358

RBM epoch 40/100, loss=0.3065

RBM epoch 60/100, loss=0.1886

RBM epoch 80/100, loss=0.1232

RBM epoch 100/100, loss=0.0298

WARNING:tensorflow:5 out of the last 36 calls to <function

TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at

0x000001E8DA5E7CE0> triggered tf.function retracing. Tracing is expensive and the excessive number

of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with

different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function

outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary

retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and

https://www.tensorflow.org/api_docs/python/tf/function for more details.

[Raisin RBM] Acc=0.8889 Prec=0.8431 Rec=0.9556 F1=0.8958

--- Raisin: Сравнение ---

Метрика	Baseline	AE	RBM	Δ _AE	Δ _RBM
Accuracy	0.8833	0.8833	0.8889	0.0	0.0056
Precision	0.8485	0.8485	0.8431	0.0	-0.0054
Recall	0.9333	0.9333	0.9556	0.0	0.0223
F1	0.8889	0.8889	0.8958	0.0	0.0069

ЧАСТЬ B. WINE QUALITY (WHITE)

--- Wine: Baseline ---

WARNING:tensorflow:5 out of the last 13 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x000001E8DC086E80> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

[Wine Baseline] Acc=0.5867 F1_macro=0.4974 F1_weighted=0.5815

--- Wine: AE Pretraining ---

[Wine AE] Acc=0.5684 F1_macro=0.4645 F1_weighted=0.5600

--- Wine: RBM Pretraining ---

RBM epoch 20/100, loss=0.0635
RBM epoch 40/100, loss=0.0435
RBM epoch 60/100, loss=0.0236
RBM epoch 80/100, loss=0.0412
RBM epoch 100/100, loss=0.0474
RBM epoch 20/100, loss=0.0316
RBM epoch 40/100, loss=0.1372
RBM epoch 60/100, loss=0.1652
RBM epoch 80/100, loss=0.1911
RBM epoch 100/100, loss=0.2924

[Wine RBM] Acc=0.5255 F1_macro=0.3900 F1_weighted=0.5127

--- Wine: Сравнение ---

Метрика	Baseline	AE	RBM	Δ _AE	Δ _RBM
Accuracy	0.5867	0.5684	0.5255	-0.0183	-0.0612
F1_macro	0.4974	0.4645	0.3900	-0.0329	-0.1074
F1_weighted	0.5815	0.5600	0.5127	-0.0215	-0.0688

ВЫВОДЫ

RAISIN:

Baseline F1=0.8889

AE F1=0.8889 (Δ =+0.0000)

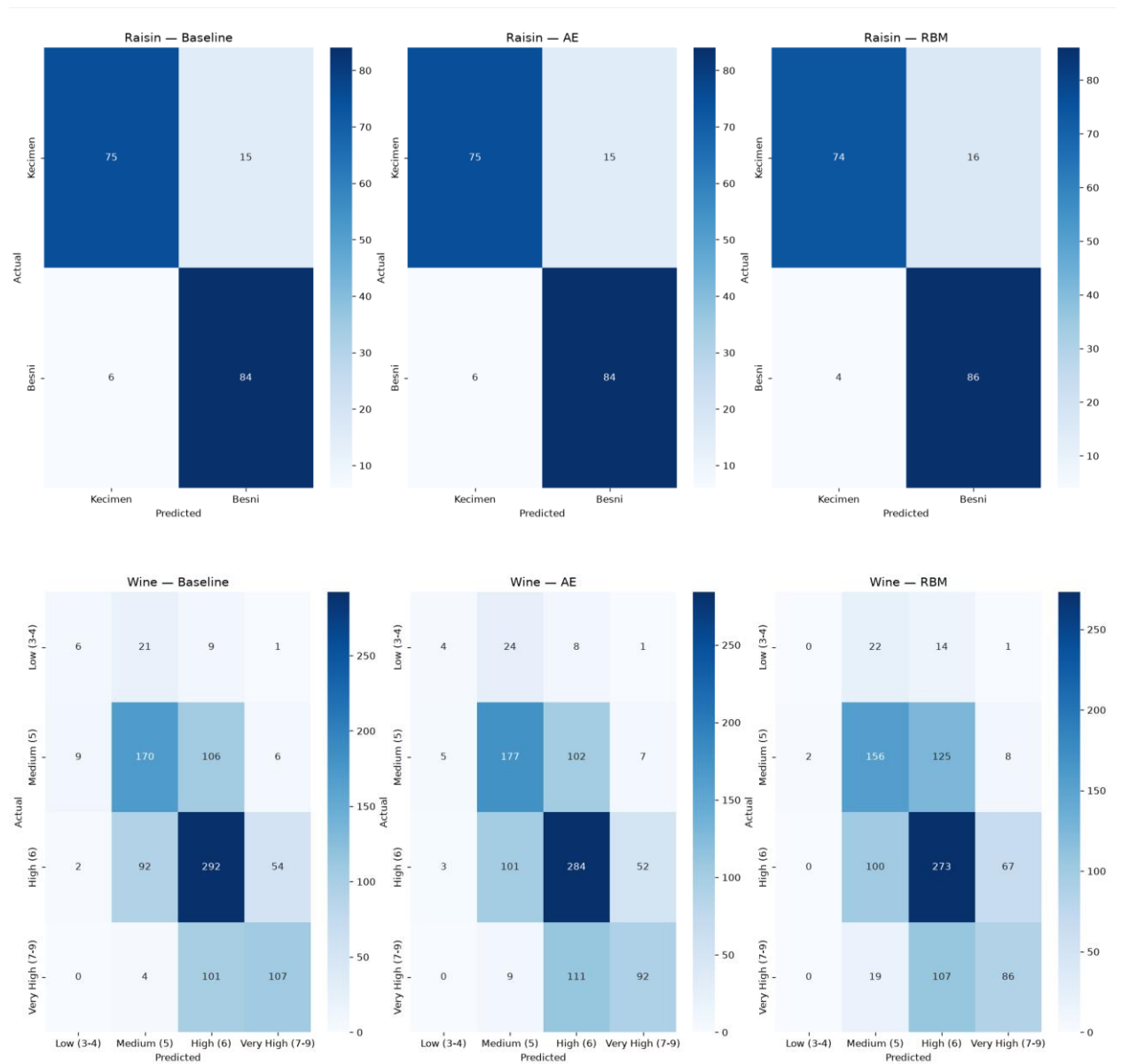
RBM F1=0.8958 (Δ =+0.0069)

WINE QUALITY (WHITE):

Baseline F1_macro=0.4974

AE F1_macro=0.4645 (Δ =-0.0329)

RBM F1_macro=0.3900 (Δ =-0.1075)



Вывод: научилась осуществлять предобучение нейронных сетей с помощью RBM.